Corpus preparation

Silence, level, separation and speaker identity — what each stage computes, and why

The algorithms behind `preprocess`, as `preprocess-core`, `audio-kit`, `burn-mdx` and `burn-campplus` implement them

---

Abstract. — A trainer eats a directory of clean per-utterance clips; a recording is almost never that. Getting from one to the other is eight transformations, of which only two run a neural network. This paper is about what each of them computes. Four ideas carry the whole phase: a **frame grid** and a percentile reading of it, which is how a recording's own quiet is measured rather than guessed; **hysteresis on the silent gap**, which is what lets a segmenter remove dead air without deleting a breath; a **complex-spectrum U-net**, which separates a voice from music by predicting phase rather than reusing it; and a **pooled speaker embedding**, which reduces a clip of any length to one vector that can be compared by cosine. The ordering of the phase follows from the first two: segmentation cuts on silence, and a continuous music bed leaves none — so a recording made over music cannot be cut into sentences at all until the bed comes off, which is a fact about the algorithm and not a preference. Sixty seconds of one real stream yields 5 segments as a mixture and 14, one per utterance, once separated. Every number below is copied from the module or harness that computes it.

Contents

1 The problem, and the eight questions

A training corpus is a directory of short clips, each one utterance, each starting and ending near the speech, at one sample rate and one channel count and one broad level. What a person hands you is a recording: minutes long, with dead air between sentences, a noise floor whose height nobody measured, possibly a music bed, possibly two people talking.

Eight transformations bridge that, and they are worth reading as questions rather than as commands:

stage	question	model
`analyze`	what **is** this recording — where is its floor, is there any dead air to cut on?	—
`separate`	which part of this signal is a voice and which is music?	MDX23C
`diarize`	which parts of this are the **same** voice as a reference clip?	CAM++
`denoise`	which part of this is a stationary noise process?	—
`normalize`	how far is this from a stated level, as one constant gain?	—
`trim`	where does the audible content start and end?	—
`resample`	what rate and channel count is this, and what should it be?	—
`clip`	where are the sentence boundaries?	—

Figure 1: The eight stages, and the question each one answers. The last column is the only place a neural network appears in this phase.

Five of those eight are, underneath, the same measurement asked differently: **where is this recording quiet?** `clip` cuts there, `trim` strips the two ends, `analyze` reports what cutting would produce, `separate` exists because a music bed makes the answer "nowhere", and `diarize`'s output is judged by whether what survives can be cut afterwards. So the silence detector is where to start, and everything after it is either a knob on that measurement or a network.

Three conventions the rest of this paper assumes. Audio crosses every boundary as **mono** `f32` in $[-1, 1]$, with one deliberate exception at Section 8. Levels are **dBFS**: $20 \log_{10}$ of an amplitude relative to 1.0, so full scale is 0 dB and everything real is negative. And every stage is a **file transformation** — audio in, audio out — so any stage's output is a legal input to any other, which is what lets the phase be reordered per corpus rather than being one pipeline.

2 Level: the frame grid, and reading it by percentile

Everything model-free here starts from one reduction of a waveform: cut it into short overlapping frames, and give each frame a single number in dBFS.

For a frame $x_0 \ldots x_{N-1}$ that number is root-mean-square amplitude,

$$\mathrm{dB}(x) = 20 \log_{10}\left(\sqrt{\frac{1}{N}\sum_{i=0}^{N-1} x_i^2} + \varepsilon\right), \quad \varepsilon = 10^{-9}$$

with the epsilon there so digital silence yields -180 dB rather than $-\infty$ — finite, orderable, and printable. RMS rather than peak because a single sample says nothing about whether a **passage** is silent, and one stray click would otherwise make a whole frame voiced.

The grid is a **≈30 ms window at a ≈10 ms hop**, both derived from the sample rate alone:

$$\mathrm{hop} = \lfloor 0.010 f_s \rceil, \quad \mathrm{win} = \max(\mathrm{hop}, \lfloor 0.030 f_s \rceil)$$

Deriving it from the rate and nothing else is what lets a measurement taken on this grid be used to set a threshold applied on the same grid — the reading and the decision cannot end up describing different framings of the recording. 30 ms is two or three pitch periods of a low voice, which is about the shortest window whose RMS is a property of the signal rather than of where the frame happened to land in a waveform's cycle.

2.1 Two percentiles, not a minimum and a maximum

Sort those per-frame decibel values and read two of them:

$$\mathrm{floor}_{\mathrm{dB}} = P_{10}, \qquad \mathrm{signal}_{\mathrm{dB}} = P_{75}$$

The 10th percentile is between-sentence dead air, room tone, and whatever the preamp contributes. The 75th is a **representative speech level**, not a peak. Their difference is the recording's signal-to-floor ratio, and it is the one number that says whether a corpus is workable at all.

Percentiles rather than extremes, because both extremes are outliers by construction: the minimum frame is whatever momentary gap the recording happens to contain and the maximum is a thump. A percentile of a distribution of frames is robust to both, and taking them over decibels rather than over RMS is the same ordering, since $20 \log_{10}$ is monotone.

Two guards make the reading honest rather than merely defined. The floor is clamped at **−120 dBFS** when a ratio is computed from it, so a digitally silent recording reports a large ratio instead of an unbounded one. And a recording of fewer than **8 frames** (≈80 ms) yields **no measurement at all** rather than a percentile of nearly nothing.

2.2 Peak, and where "clipped" is

The other level statistic is the plain maximum absolute sample, and the only subtlety is what counts as full scale. A sample is treated as clipped at **0.999**, not at 1.0, because 16-bit full scale decodes to $32767/32768 = 0.99997$ — an exact test against 1.0 reports nothing at all on the commonest source of clipping there is.

3 Silence: hysteresis segmentation

The segmenter turns a mono recording into voiced `[start, end)` sample ranges. Its design rule is one sentence: **energy is used solely to locate long silent gaps, never to gate quiet-but-present sound.** In soft, breathy, close-mic material the quietest passages are content, and a plain energy gate — keep frames above a threshold, discard the rest — deletes precisely them.

The algorithm is four steps.

1. **Frame and threshold.** A frame is voiced when its dB value is $\geq$ the silence floor. This produces raw voiced runs.
2. **Merge on hysteresis.** Two consecutive runs separated by a gap **shorter** than the minimum-gap parameter are one run. This is the whole design: a gap has to be both below the floor **and** long enough to be a sentence boundary, so an internal pause and a soft tail stay inside the clip.
3. **Pad.** Each surviving run is widened by up to the pad parameter into the bordering quiet, so an onset is not clipped and a breathy tail is not cut off. A pad is bounded at the **midpoint** of the gap to the neighbouring run, so two neighbours can never both claim the same silence and overlap.
4. **Cap and filter.** A run longer than the cap is split recursively at its quietest interior frame; a run shorter than the minimum length is dropped.

parameter	default	decides
silence floor	−40 dBFS	what counts as quiet at all
minimum gap	0.3 s	how long quiet must last to be a **boundary** rather than a pause
minimum length	1.0 s	the shortest kept segment
cap	0 (off)	a hard maximum; longer runs split at their quietest interior frame
pad	0.15 s	how much bordering quiet each kept segment keeps

Figure 2: The five parameters, their defaults, and what each decides. `trim` uses the first and last only — see Section 4.

Two details of the splitting step are worth having. The quietest interior frame is searched only within a margin of each end — the largest of a quarter of the cap, one hop, and the minimum length — so both halves make progress and neither is a fragment the next filter would silently discard. And ties are broken **towards the midpoint** of the run, with a relative tolerance, so a genuinely flat stretch splits evenly instead of at whichever frame won by a rounding error.

The minimum-length parameter has a floor under it that the segmenter cannot see from where it lives: `rvc-train` draws 0.48 s windows, and a clip shorter than one window is decoded, feature-extracted and **then** discarded. Below 0.48 s the parameter buys nothing and costs the analysis of every fragment it lets through. That number is deliberately not a constant in the shared crate — a crate that knew a trainer's window size would be depending on an engine — so it is stated rather than enforced.

3.1 When is a boundary final?

The same algorithm has an online form: push audio as it arrives, take back each segment as soon as it provably cannot change again. That "provably" is the interesting part, because it is what makes the streaming and batch paths cut in **identical** places rather than trading accuracy for latency.

A voiced run's range can still move for exactly two reasons: the silence after it might turn out to be an internal pause, so the run grows; or the next run might start close enough to bound this one's trailing pad at the midpoint. Both stop being possible once enough silence has followed. The settle time is

$$\text{settle} = \max(\text{minimum gap}, \lceil (2 \cdot \text{pad} + \text{win}) / \text{ hop} \rceil) \text{ frames}$$

— the first term retires the merge, the second puts any future run more than two pads away, so the midpoint bound can no longer bind. Past that point the online segmenter emits the range, and it is

the range the whole recording would have given. A property test pins the two paths to identical cuts, which is what makes that claim checkable rather than argued.

There is exactly one divergence, and it is in the cap: speech that never falls silent is force-cut by the online path at the quietest frame **in the window it has**, where the batch path knows the run's full length and balances the split across it. Cutting late is the price of not buffering the recording.

This is also why speech recognition can transcribe a ten-minute file incrementally rather than reading all of it first: bounded lookahead means segmentation is an online problem, not a whole-file one.

3.2 Measuring the floor instead of guessing it

−40 dBFS is a number nobody can estimate about a recording they have not analysed, and on breathy close-mic material getting it wrong is exactly the failure this phase exists to prevent. The percentile pair of the previous section is enough to derive it:

$$\text{silence}_{\text{dB}} = \min(0, \text{floor}_{\text{dB}} + \min(8\ \text{dB}, 0.4 \cdot (\text{signal}_{\text{dB}} - \text{floor}_{\text{dB}})))$$

Both constants answer a different failure, and the second exists because the first cannot be the whole story.

- **8 dB above the floor.** Room tone is a distribution, not a level, and the measured floor is the **middle** of the dead-air frames. A threshold sitting on it would call half the dead air voiced, and that dead air would survive into the corpus.
- **...but never more than 40% of the way to the speech.** A close-mic breathy take has almost no range to spend a fixed margin in. On this repository's own corpus, rebuilt into a raw recording — ten sentences with a second of the corpus's own room tone between them — the floor measures −51.4 dBFS and the speech −41.6: a **9.8 dB** gap, in which a flat 8 dB margin lands 1.8 dB **under the voice**. Taking a fraction of the measured gap instead makes the threshold structurally unable to reach the speech, and both failures are then bounded from one measurement rather than by two unrelated constants.

0.4 is where a sweep put it. Every value from 0.3 to 0.6 recovers all ten sentences of that recording — against **five** for the fixed −40 dBFS — but they differ in what they keep of the soft tails: 28.5 s at 0.3, 27.9 s at 0.4, 27.0 s at 0.5 and 25.9 s at 0.6, out of a 29.7 s ceiling (all speech plus the full pad on each side). The fixed floor keeps 8.2 s. 0.4 keeps 94% of the ceiling while still sitting 3.9 dB clear of the tone.

It is a whole-signal statistic, and that decides where it can live. The online segmenter never sees the whole signal — that is the point of it — so measuring inside it would give the streaming path a floor that moved as audio arrived, and the two paths would stop cutting in the same places. The measurement therefore happens once, before segmentation, and hands both paths the same concrete number. The honest cost is that a true stdin filter has no whole signal to measure, so this is reachable only where the audio is already in memory. It is opt-in for a second reason that is policy rather than arithmetic: every corpus prepared so far was cut at the fixed floor, and deriving one by default would silently re-cut all of them.

4 Two verdicts on the interior: `clip` and `trim`

These two stages share every line of silence detection and differ in exactly one respect — what they do with the gaps in the **middle** of a recording.

`clip` cuts there: each input becomes one file per sentence. This is the stage the trainers want first, and the reason is specific rather than aesthetic. `rvc-train` draws random 0.48 s windows uniformly

across each corpus file, so a raw recording that is half dead air trains the generator on silence half the time, and it collapses to silence.

`trim` keeps them, and it is implemented as **the same segmenter with the boundary test disabled**: run it with the minimum gap set longer than the recording, the minimum length at zero and the cap off, and no interior gap can ever be a cut point, so there is exactly one merged run by construction. Its `[start, end)` is the first voiced sample to the last, padded, and everything between is written through untouched — pauses included, never a split. Two consequences fall out of that construction rather than being coded: it carries only the two parameters that mean something without cutting (the floor and the pad), and a recording with nothing above the floor produces no run at all, which is written through whole rather than emptied. A file whose floor was simply set too high is not silently deleted.

Reusing one detector rather than writing a second is the point. A second silence detector would be a second set of answers to "where does this sentence end", and the hysteresis that keeps a breathy tail inside a clip is the whole reason the first one is written as it is.

5 `analyze`: what the recording is

Every other stage has at least one parameter whose right value depends on the recording. `analyze` is the half that shows the user what the recording is: it loads no model, writes no audio, and therefore costs a decode and nothing else.

It also **measures nothing of its own**. The floor is the percentile pair of Section 3.2, and the clip counts come from **running** the segmenter rather than from a model of it. A stage that predicted the segmenter's output would drift from it the first time either changed, while still printing numbers that looked like the truth.

5.1 Two floors, because a suggestion has to be checked

Each file's report holds the segmenter's verdict **twice**: once at the floor the user asked for, and once at the floor measured from the recording. Reporting only the second would make the suggestion an assertion; running the segmenter at it is what turns it into a measurement — and it is the only way to detect the case that matters most.

A recording with no dead air at any floor is that case, and it is what speech over a continuous music bed looks like from a segmenter's side: the floor sits under the music, no frame is quiet, and no threshold rescues it, because the quiet is not there to find. The condition is reported when a recording is longer than **15 s** and the better of its two floors still leaves under **2%** of it silent. Both bounds are needed: under 15 s a gapless file is just a short one, and the **output** of `clip` is gapless files of a few seconds, so an already-sliced corpus would otherwise report every clip as pathological. When it holds everywhere, a suggested floor is **withheld**, because a floor that cannot work is worse than none — the answer is separation, not a threshold.

Beyond the two verdicts, a report carries duration, the floor/signal pair, peak in dBFS, a count of clipped samples, a duration histogram of the clips that would be produced, and the silence ratio — the fraction of the recording segmentation would discard. There is no integrated-loudness column: peak, floor and signal-to-floor ratio answer "is this corpus usable", and loudness is a question about **matching** recordings, which is Section 7′s.

6 `denoise`: averaging over time, not over frequency

Hiss that survives into the training clips is hiss the model learns to reproduce, so removing it once beforehand is both cheaper and better than removing it from every conversion afterwards.

The filter is **non-local means** over the waveform. For each short patch of audio it searches a research window nearby **in time**, weights every candidate patch by how similar it is to the current one, and replaces the sample with that weighted average. Stationary hiss looks the same everywhere, so it finds many close matches and averages away; an ever-changing breath texture finds few, so its own sample dominates its average and it survives.

That property is the entire reason for the choice. Breath is spectrally almost indistinguishable from hiss — both broadband, both low-level — so a spectral-subtraction de-noiser, which decides by level and frequency, has no feature that separates them: tuned to remove the hiss it removes the breath, and leaves warbly musical noise where it was. Non-local means keys on **repetition** instead, which is the one axis on which the two differ.

Three parameters, and they are the algorithm's own: patch duration (2 ms — the unit compared for self-similarity, small enough to keep fine detail), research window (6 ms — how far in time it looks, which must exceed the patch and also sets the filter's latency), and strength (0.008 — how aggressively similar patches are pooled). On a breath-over-hiss signal typical of soft close-mic speech that setting removes about **75%** of the hiss while preserving content energy and high-frequency crispness to within about **1%**, and runs at roughly **12× realtime**, which is what lets the same filter sit in a realtime conversion path.

Two implementation properties matter to the result. Each file gets its own filter graph, so the research window never spans two recordings — a patch from one voice can never be averaged into another. And the graph **fails open**: if ffmpeg cannot build it, audio passes through unchanged with a warning rather than being dropped.

7 `normalize`: one gain, held constant

One file in, one file out, at a gain that is **constant over the whole file**. That is the property the stage is built around rather than an implementation detail: a corpus of soft, breathy material is mostly quiet on purpose, and anything that moved the gain about while it played would flatten exactly the content this toolkit exists to preserve. Nothing here is a compressor, and that is not an omission.

7.1 Two targets, answering different questions

Peak puts the loudest sample at a stated fraction of full scale, 0.95 by default. It is exact, instant, and blind to everything but that one sample, so a single stray thump decides the gain for a whole recording. The 5% of headroom is there because a later resample's interpolation can overshoot the samples it interpolates between.

Loudness is EBU R128 integrated loudness in LUFS, which is what "as loud as each other" means to a listener. Three things distinguish it from an RMS average, and all three matter for speech:

- **K-weighting.** The signal is filtered by a shelf plus a high-pass before power is taken, approximating how the ear weights frequency — so rumble and hiss contribute far less than the voice does.
- **400 ms blocks, 75% overlap.** Loudness is a property of a passage, not of a sample.
- **Gating.** Blocks below an absolute gate of −70 LUFS are discarded, and then blocks more than 10 LU below the mean of what survived are discarded too. This is why the silence between sentences does not drag the reading down, and it is the reason two takes of different sparseness can be matched at all.

Peak when you want a known headroom, loudness when you want two takes to sit at the same level. They are alternatives rather than a default plus an override, because **which** target is the question being asked.

7.2 The measurement is ffmpeg's; the gain is ours

The obvious filter is `loudnorm`, and it is wrong twice over. In single-pass mode it is a **dynamic** normalizer — it compresses and true-peak limits, which is precisely the processing a breathy corpus must not get — and its linear mode needs measured values it will only ever print to a log, which an in-process filter graph cannot read. It also renegotiates its output to 192 kHz.

`ebur128` has neither problem: it passes the audio through untouched and injects the running measurement as frame metadata, so ffmpeg does the standard-compliant part and the gain stays one multiply. The reading is taken **after** the flush, not before — the integrated value is a running one, so the figure covering the whole recording is on the last frame out.

7.3 Two edge cases that would otherwise be silent

The gate parses. ffmpeg reports the absolute gate itself rather than $-\infty$ when nothing clears it, so digital silence measures exactly -70.0 LUFS: finite, parseable, and normalised **from** as though it were a very quiet recording. Two seconds of zeros against a -23 LUFS target would receive $+47$ dB of gain and come out as amplified nothing. Both spellings of "nothing cleared the gate" therefore have to be tested for, and the second is the one that looks like a reading.

A recording shorter than one 400 ms block has no integrated loudness at all — not a quiet one, none. That is reported as its own answer rather than papered over with a peak fallback the user did not ask for.

8 `resample`, and the one stereo exception

Every other stage resamples on the way through, because every one of them decodes. This is that operation with nothing else attached, so a corpus arriving as mp3, m4a, flac and opus at three different rates can be given one normalising step.

Rate conversion is ffmpeg's, and the part worth knowing about it is the low-pass. Halving a sample rate halves the Nyquist frequency, and any energy that was above the new one does not disappear — it **folds back** below it, as an inharmonic mirror image that no later stage can distinguish from signal. So a downsample is a band-limiting filter followed by a decimation, never a decimation alone, and the filter is where the quality of a resampler lives. Upsampling has the mirror problem and the same answer: interpolate, then remove the images the interpolation creates. This is also the overshoot the peak target of Section 7 leaves headroom for — a band-limited reconstruction can exceed the largest sample it was built from.

Mono by default, because mono `f32` is what crosses every crate boundary here and a training corpus has no use for a second channel. A stereo input is folded to $(L + R)/2$; a mono input written as stereo goes to both channels, so the output is uniform whatever went in.

The exception is `separate`, which writes **44.1 kHz stereo** on purpose: MDX23C is stereo-native, and inter-channel difference is one of the two cues it separates on, so folding its input to mono discards half of what it works with. That single decode-stereo/write-stereo path is the only place the workspace's mono rule is broken, and keeping it that way is the property — a mono `resample` after a `separate` is a legitimate choice, since everything downstream decodes mono anyway, but it should be a choice rather than a surprise.

9 `separate`: a complex-spectrum U-net (MDX23C)

Source separation asks: given a mixture, which part of it is a voice? The model is MDX23C — TFC-TDF-UNet v3 — and the interesting thing about it is **what it predicts**. It does not predict a mask over a magnitude spectrogram, which is the classical formulation. It consumes the complex spectrum,

predicts a complex spectrum per stem, and inverts it. Phase therefore travels **through** the network rather than being reused from the mixture, which is what allows the stems to sum back to the mixture rather than merely to resemble it.

The architecture **is** the checkpoint: `dim_f`, `n_fft`, the channel widths and the block counts all differ across the MDX family, so a port is written against one file. This one is `MDX23C-8KFFT-InstVoc_HQ.ckpt` (448 MB, 319 tensors, a bare `state_dict` at the root), whose config is pinned as a constant rather than parsed, so a disagreeing checkpoint fails as a shape mismatch rather than being silently accommodated. Every field is confirmed by a tensor in the file: `first_conv.weight` is `[128, 16, 1, 1]`, giving 128 channels and 16 input channels; `bottleneck_block.blocks.0.tdf.2.weight` is `[8, 32]`, which is five halvings of 1024 and therefore five scales; `final_conv.2.weight` is `[32, 128, 1, 1]`, so two stems. The stem **order** — vocals first — comes from the config's instrument list and is not recoverable from the weights at all.

9.1 The front end

The transform is a plain complex STFT, and every parameter of it is a decision the network was trained under:

- $n_{\text{fft}} = 8192$ at 44.1 kHz — a 186 ms window, which is long. Music separation wants frequency resolution (5.4 Hz per bin) far more than it wants time resolution, because the thing being separated is largely a difference in harmonic structure.
- `hop` $= 1024$, so $4\times$ overlap.
- **Periodic** Hann, matching `torch.hann_window(periodic=True)`: the denominator is n_{fft}, not $n_{\text{fft}} - 1$. The symmetric window would shift every sample by half a bin's worth of taper and show up as a quiet broadband reconstruction error rather than as anything that looks like a bug.
- `center = true` with **reflect** padding — torch's `stft` default — so `samples` yields samples / hop $+ 1$ frames and the inverse returns hop $\cdot$ (frames $- 1$) samples.
- Only $\dim_f = 4096$ of the 4097 bins are kept; the rest are dropped on the way in and zero-filled on the way out. A round trip is therefore lossy by construction, which is upstream's behaviour and not an artefact.

That frame arithmetic is what makes the chunk length a **consequence**: hop $\cdot$ ($\dim_t - 1$) $= 1024 \cdot 255 = 261{,}120$ samples, 5.92 s, is exactly 256 frames.

The transform is host-side (`rustfft`) rather than a Burn graph, and the reason is size: a DFT-basis matmul at $n_{\text{fft}} = 8192$ is a $8192 \times 4097 \times 2$ constant — 268 MB — for something a radix-2 FFT does in microseconds. A differentiable STFT has to be a graph; an inference-only one does not.

9.2 Complex as channels, the subband fold, and the transpose

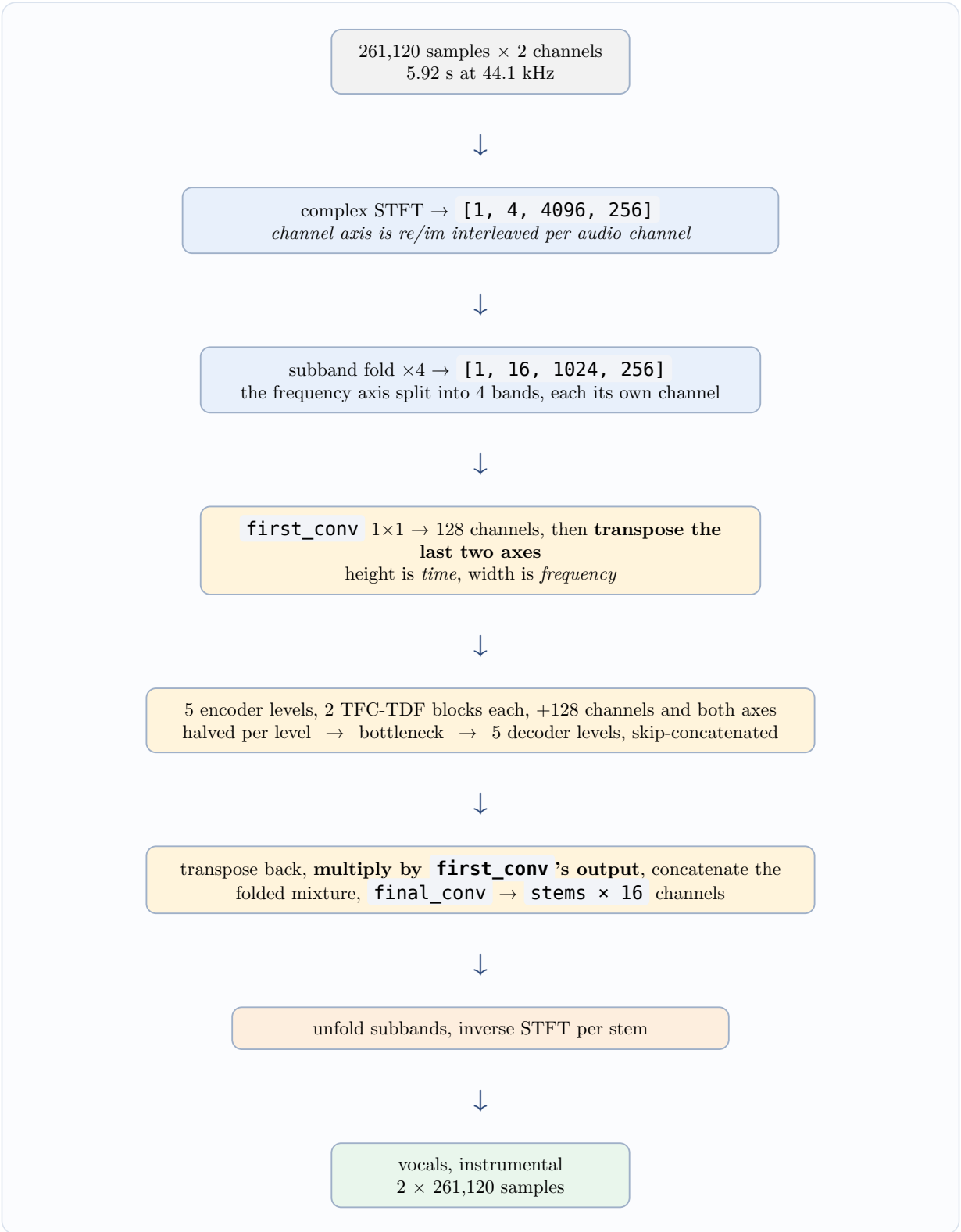

Figure 3: One chunk through the network. Batch is 1 throughout. What changes at the transpose is which axis the `Linear` inside each block will act on.

Complex as channels is the first idea: real and imaginary parts become separate image channels, interleaved per audio channel — channel $2c$ is audio channel c's real part and $2c+1$ its imaginary part. A stereo mixture is therefore a 4-channel image of frequency × time, and a 2-D convolution

over it sees re and im of both channels at once, which is how phase becomes something the network can act on rather than something it must preserve.

The subband fold is the second: the frequency axis is split into 4 contiguous bands — band 0 is the lowest 1024 bins — and each becomes its own channel, so the network sees a shorter, wider image, $[1, 16, 1024, 256]$. Convolutional weight sharing along frequency is a poor assumption in the first place (a harmonic at 100 Hz and one at 10 kHz behave nothing alike), so trading frequency extent for channels both fixes that and makes the image cheap.

The transpose is the whole trick. The tensor is `[batch, channels, height, width]` throughout, but which axis is which changes exactly once: the last two are swapped before the encoder and back after the decoder, so **inside every block the height is time and the width is frequency**. That is what puts the frequency axis last for the fully-connected layer inside each block — Burn's `Linear`, like PyTorch's, only ever acts on the last axis — and it is the single detail that makes a transposed port load at 100% and separate nothing.

9.3 The block, and the head

One **TFC-TDF block** is a residual pair with a 1×1 shortcut:

$$\text{block}(x) = \text{shortcut}(x) + \text{TFC}_2(y + \text{TDF}(y)), \quad y = \text{TFC}_1(x)$$

- **TFC**, time-frequency convolution, is `norm → GELU → 3×3 conv`. Local, in both time and frequency.
- **TDF**, time-**distributed** fully-connected, is `norm → GELU → Linear(f → f/4) → norm → GELU → Linear(f/4 → f)`, both linears bias-free. It is applied per time frame across the **whole** frequency axis, so it is the block's only global operation — it can relate a fundamental to its harmonics several thousand bins away, which no 3×3 convolution can.
- The TDF is a residual on the TFC's output, not a second path from the block input. Reading it the other way round is a plausible misreading of three consecutive statements and changes what the bottleneck sees.

Two orderings in that block are load-bearing and invisible to a coverage report. **Normalisation and activation come before the convolution**, so a port that reads `conv → norm → act` off habit loads perfectly and computes a different function. And the norms are **instance** norms — per sample, per channel, over both spatial axes — which the checkpoint is the only thing that says: every norm carries `weight` and `bias` and no running statistics. Normalising by a stale running mean instead of the sample's own is silent; the output stays finite and the separation just gets worse.

The shortcut is present on **every** block, including those mapping a channel count onto itself, which is the opposite convention from RMVPE's `ConvBlockRes` (where it exists only when the width changes). The checkpoint settles it: a `shortcut.weight` for all 110 blocks.

The head is not just a projection. The decoder's output is multiplied elementwise by the first convolution's output, and the folded mixture is concatenated beside it before the final 1×1 — upstream's comment is "reduce artifacts". Two consequences follow. The network keeps a direct path from its input to its output, which is why the stems partition the mixture so precisely. And **the network is not scale-invariant** — its instance norms are, but a multiplicative gate and a concatenated raw mixture are not, so the two paths scale differently. That is why the caller has to feed it audio at a level it was trained on, and it is the subject of the next section.

9.4 The chunk, the seams, and the level

The network's unit of work is one chunk, and it knows nothing about a longer recording — deliberately, because separation runs on songs and decoding a ten-minute stereo file into memory before the first forward pass costs several hundred megabytes. Even one chunk is not small: the first encoder level holds $[1, 128, 256, 1024]$ activations, 134 MB each, and the widest decoder concatenation is 268 MB. A 6 GB GPU fits it; a pure-CPU backend will run it and should not be asked to. One chunk costs about 1 TFLOP, measured at **0.83 s** on LibTorch/CUDA, so a real recording separates at roughly 3.5× realtime at 50% overlap.

Everything longer than a chunk is the caller's, and there are two things to get right.

Overlap-add. Consecutive chunks are windowed by a periodic Hann at a half-chunk hop, which sums to one across the hop — the constant-overlap-add condition. The accumulated weight is divided out **explicitly** rather than relied upon, so a changed hop degrades the blend gracefully instead of introducing a gain ripple. One region needs special handling and the failure there is silent and one sample wide: nothing overlaps the **leading** half of the first chunk, so its accumulated weight is the rising half of the window alone, which is exactly zero at frame 0 — dividing a zeroed sample by a guarded zero writes a zero where the recording's first sample should be. That leading half is therefore windowed by **one** instead: it is covered by a single pass, and the honest reconstruction of a region covered once is the model's own output. The end needs no such treatment, because the last pass's frames are always overlapped by the descending tail of the one before.

Level. Because of the head, the mixture is scaled so its peak lands at **0.7** before the first chunk reaches the model, and the gain is undone before anything is written. Two details make that more than a detail. It has to be known up front — a gain that changed part-way through would be a seam no crossfade can hide — which is why the stage makes a first pass over the stream for the peak alone, streamed like the second so it costs a decode rather than a copy of the recording. And the argument runs **downward** as well as upward: a recording peaking at −30 dBFS is scaled **up** to 0.7 rather than left alone. Undoing the gain afterwards is equally load-bearing: a stem left at the model's working level is uniformly attenuated or boosted against its own mixture, and every level anyone reads off it later — "how much quieter is the bed in the speech gaps" — would be wrong by exactly that gain, which reads as a separation result.

9.5 What it does not do

It separates **voice from music**, not one voice from another. Every speaker in the recording lands in the vocals stem, including a singer on the backing track, so a streamer talking over somebody else's **singing** still gets both. Filtering by **who** is speaking is a different question and a different model.

> **A note on the older MDX-Net v2 models.** They are what UVR5 shipped first, and they are published as ONNX only, as BN-fused exports: roughly half of every file's 220 initializers carry no name at all, because `Conv → BatchNorm` was constant-folded and `Linear(bias=False)` became a `MatMul` over a transposed anonymous constant. One architecture even has two naming schemes on disk, so no remap is right for both. They can be **run** — the STFT above is parameterised by $(n_{\text{fft}}, \text{hop}, \text{dim}_f)$ precisely so it can front either — but a coverage number measured against a port of them would be fiction. Two facts a caller wiring that path needs: UVR keys its hyper-parameter table by the MD5 of the **last 10,240,000 bytes** of the `.onnx`, and in that table `mdx_dim_t_set` is an **exponent**, so 8 means 256 frames.

10 `diarize`: a pooled speaker embedding (CAM++)

Separation cannot answer "whose voice is this", because it asks "is this a voice". Telling two speakers apart needs speaker **identity**, and the standard formulation is an embedding: a network that maps a clip of any length to a fixed vector — here 192 dimensions — trained so that two clips of the same speaker land close together under cosine similarity and two speakers land apart.

Everything the stage does follows from that. A reference clip of the voice to keep is embedded once; the recording is cut into fixed windows, each embedded whole; each window's cosine against the reference decides whether it is kept; runs of kept windows become the written segments. This is **target-speaker extraction** rather than clustering — it needs a sample of the voice you want, which a user extracting their own speech from a stream always has, and it is far more robust than discovering how many speakers a recording contains.

10.1 The filterbank is part of the contract

CAM++ eats a **Kaldi filterbank at 16 kHz** — 80 bins, 25 ms window, 10 ms hop, mean-normalised over time. Not the 22.05 kHz 80-band mel a vocoder speaks. Both are "an 80-band mel" by shape, so the wrong one loads, runs, and produces a plausible vector from a distribution the network has never seen. The differences are each a place where substituting one for the other computes something else:

- Per-frame DC removal, then a **preemphasis of 0.97** with a replicate pad at the frame's left edge — so the first tap keeps $1 - 0.97$ of itself rather than being high-passed against a neighbour it does not have.
- The **Povey window**, $(0.5 - 0.5\cos(2\pi n/(N-1)))^{0.85}$: a **symmetric** Hann raised to 0.85, which reaches zero at both ends where a periodic Hann does not.
- The 400-sample window is zero-padded **on the right** to 512 before the transform. Right, not centred — a centred pad would rotate every frame's phase.
- `snip_edges` framing: no padding at either end, so the frame count is $1 + (\text{samples} - 400)/160$ and the final partial window is dropped.
- A **power** spectrum, and Kaldi's own mel scale $1127\ln(1 + f/700)$ with triangles laid out evenly in that domain from 20 Hz to Nyquist — not Slaney, and not area-normalised.
- The Nyquist FFT bin carries **no weight at all**: Kaldi builds the bank over half the padded window — 256 bins — and torchaudio pads a zero column onto the right to reach the 257 an `rfft` returns.
- The log is floored at `f32::EPSILON`, which is what `torch.finfo(torch.float).eps` hands `torch.max`.

Waveform scale, notably, does **not** matter. Kaldi proper expects samples scaled like int16 and upstream hands it a $[-1, 1]$ float waveform — but a gain of a adds $2\ln a$ to every bin of every frame, and the mean subtraction removes exactly that. The one place the scale is visible is the epsilon floor, so the port follows upstream's convention rather than Kaldi's: the floor has to sit where the released model saw it.

That mean subtraction is part of the front end, not of the model, and it is the trap the crate is arranged around. Upstream's inference writes the encoder's input in two lines — the filterbank, then subtract the per-clip mean over time — and a port written by reading the **model** is faithful and still wrong, because CAM++ itself never normalises. The failure is silent: the embedding stays finite, stays repeatable, and quietly starts keying on the recording's channel rather than on the speaker, which looks like a threshold that needs tuning and not like a bug.

10.2 The network

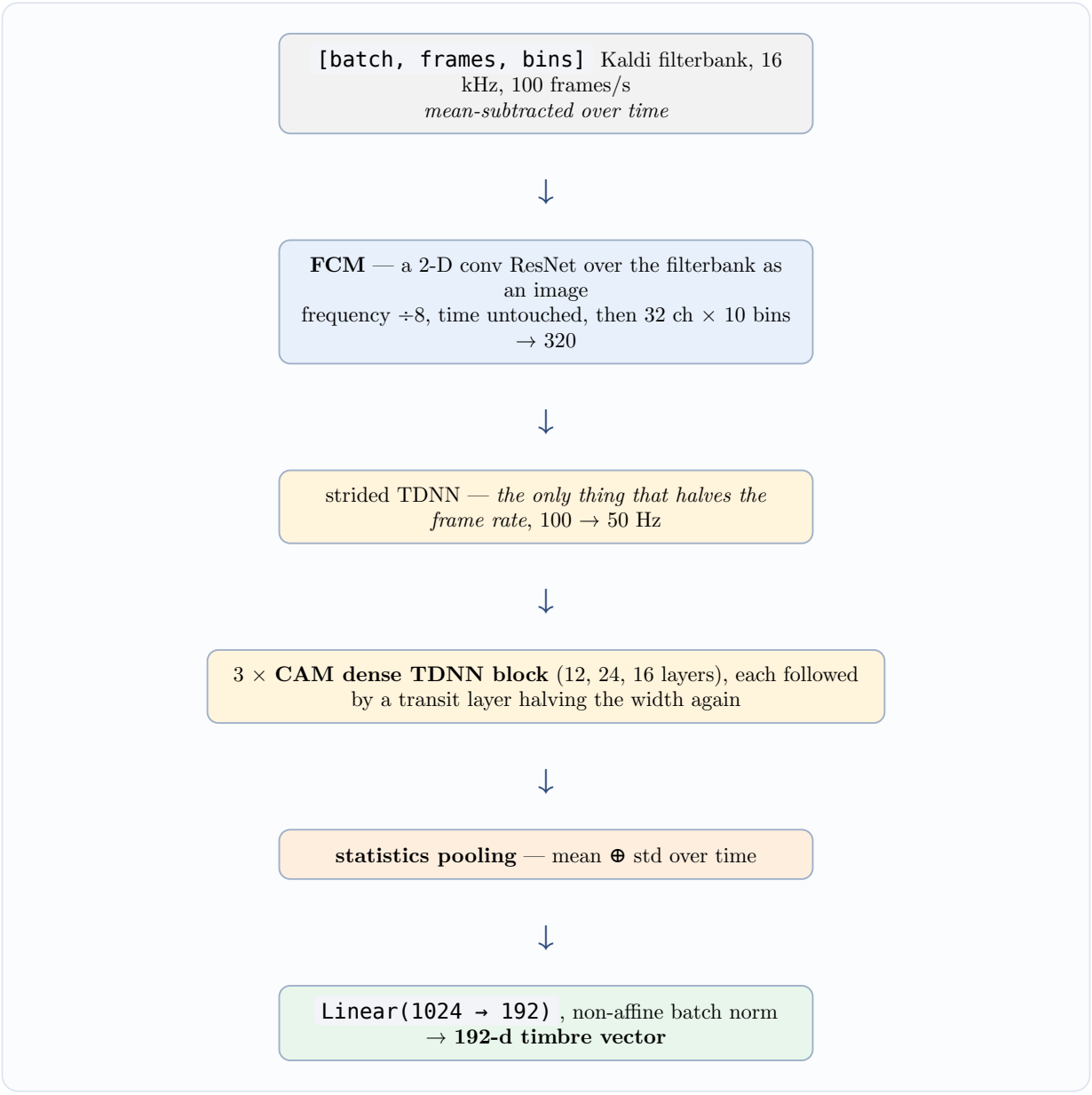

Figure 4: CAM++, 80 filterbank bins in, a 192-dimensional vector out.

A **TDNN** is a 1-D convolution over time with dilation — the classic speaker-embedding front end, because it grows a receptive field over a whole utterance cheaply. **Dense** means every layer's output is concatenated onto its input, so a block's width grows by a fixed rate per layer and the transit layer after it halves the width again.

Context-aware masking is the "CAM", and it is the piece worth reading twice. Each layer computes a local convolution and multiplies it by a gate derived from two pooled summaries: the whole utterance's mean, plus a 100-frame segment average broadcast back over time. Every frame is therefore scaled by what its neighbourhood and the utterance as a whole look like — which is how a speaker-level network suppresses content-level detail, since what is being looked for is exactly the part of the signal that does **not** change as the words change.

Statistics pooling — concatenate the mean and the standard deviation over time — is what makes a clip of any length produce a fixed vector, and therefore what makes cosine comparison possible at all. Note what it does **not** make length-independent: the vector's **shape** stops depending on length,

but the cosine distribution two clips land in does not, which is why a threshold has to be calibrated at the window length it will be used at (Section 10.4).

The final norm always normalises by the checkpoint's stored running statistics rather than the batch's. That is what makes batching many windows in one forward pass **safe** rather than merely fast: with per-item pooling and no batch statistics anywhere, an embedding cannot depend on what else was in the batch. A batch-statistics norm would have made the whole stage silently order-dependent. The clips in one batch must be the **same length**, though, because they are stacked into one tensor — and padding a short one out would put its silence inside the per-clip mean and then pool over it, which is wrong rather than merely different, so a short tail is dropped instead.

Each window is embedded from its own filterbank, never from a slice of the file's. The front end subtracts the per-clip mean, so a mean taken over a whole recording of two speakers and a music bed would normalise every window against a distribution none of them has. Computing one filterbank per file and slicing frames out of it is roughly **fifty times cheaper** and silently wrong, which is exactly why it is worth naming: it is what an optimiser reaches for.

10.3 The output is window-quantised

Segment boundaries land on the window hop, not on the sample where one voice stopped, because **a window that spans a speaker change belongs to whoever dominates it**. Two consequences worth expecting rather than discovering: a retained segment can carry a fraction of a second of the other voice at each end, and a one-word interjection shorter than a window will not be removed on its own.

The window length is the real trade. Too short and the embedding is noisy — CAM++'s context-aware mask pools over 100 of its own 50 Hz frames, so below about 2 s its two context scales collapse into one and the "utterance-level" summary stops being one. Too long and a speaker change hides inside a window. The hop is the output's resolution: halving it doubles the work and halves the quantisation.

10.4 Where 0.55 and 3 s came from

The calibration harness measures **the same quantity the stage compares**: one whole-reference embedding against one fixed-length **window** embedding, not clip against clip. A threshold calibrated on sentence-length clips would be a threshold for a different measurement.

The numbers below are from one 60 s excerpt of a stream with a song playing under it, separated first. The **vocals** stem stands in for the target speaker's material and the **instrumental** stem — the same seconds with him removed, leaving the music and the singer — for what has to be rejected. The reference is 14 s of the same speaker from a different part of the same recording.

window	**target** p5 / p25 / median	**reject** median / p75 / p95
1.5 s	0.215 / 0.456 / 0.562	0.362 / 0.449 / 0.609
3 s	0.374 / 0.568 / 0.628	0.411 / 0.464 / 0.674
5 s	0.488 / 0.625 / 0.660	0.430 / 0.510 / 0.722

Figure 5: Cosine against the reference, by window length. At 1.5 s the two distributions do not separate at all — the target's lower quartile sits **below** the rejected material's upper one, which is the 2 s pooling window showing up as a number.

5 s separates marginally better than 3 s and quantises every boundary five seconds wide, so **3 s** is the default window, with a 1 s hop, a 0.55 threshold and a 1 s minimum segment. At 3 s, 0.55 keeps 80% of the target's windows and rejects 86% of the music-and-singer ones.

The reading to take from a harness like this in general is **the gap**: the same-speaker 5th percentile against the different-speaker 95th. If those overlap, no threshold separates those two voices and the honest report says so instead of picking one. Two limits bound this particular measurement from above, and both are properties of the data rather than of the model. The reference and the material share a session, so the same microphone and the same room — and CAM++'s known failure is keying on the **channel** rather than the speaker, which flatters a same-session measurement. And the rejected material is a music bed with a sung voice in it rather than a second person talking, because no clean recording of a second speaker was available.

There is no level gate, and that is a measurement rather than an oversight. A window with no voice in it embeds to something arbitrary, so the obvious guard is an RMS floor. It is unnecessary because the threshold already covers it: over that excerpt the near-silent windows scored **0.20–0.37** against the reference, far below any threshold that keeps the target, and a −40 dBFS floor removed 3 of 58 windows that were being dropped anyway. A floor would be a second knob agreeing with the first.

11 How you know any of this works

Two kinds of check run against the two networks, and conflating them is the error this section exists to prevent.

Coverage proves a tree, never an arithmetic. MDX23C loads at 319 applied / 0 missing / 0 unused, CAM++ at 815 / 0 / 122 — the 122 being one training counter per norm, which inference never reads. Both numbers say the module tree matches the checkpoint, and both would be **unchanged** by the U-net running with frequency as the image height, by a batch norm where an instance norm belongs, or by a filterbank missing its mean subtraction. So each network needs a second check that exercises arithmetic on real audio.

11.1 Separation, synthetically: the check that proves the port

Mix a known voice with a known bed and every reading has a reference. Measured on LibTorch/CUDA over six clips of this repository's own corpus against a synthesised organ chord at a 0 dB mix:

reading	value
partition — the two stems summed, against the mixture	**41.5 dB** SI-SDR
solo voice in — vocals / instrumental rms	0.0516 / 0.0299 (**4.7 dB**)
solo bed in — vocals / instrumental rms	0.0226 / 0.0526 (**7.3 dB**)
SI-SDR of the vocals stem against voice / against bed	−0.25 / −0.54 dB
envelope corr, vocals stem against voice / bed	0.678 / 0.645
envelope corr, instrumental stem against voice / bed	0.508 / 0.583

Figure 6: The synthetic mode. **Read the first row first.**

The two stems reconstruct their input to 41.5 dB, and the solo probes split **in opposite directions** depending on which source went in. Together those say the forward pass is coherent and content-dependent — a transposed U-net, a batch norm where an instance norm belongs, a mis-scaled block or a scrambled stem axis destroys one or both, because nothing downstream re-imposes either property.

What it does not establish is separation quality, and the numbers are honest about that: 4.7 dB of rejection is far below the 20 dB-plus a vocal separator manages on the material it was trained for, and every mixture-level SI-SDR row sits within a decibel of doing nothing. The reason is the input rather than the port — MDX23C was trained on **sung** vocals inside real productions, and this feeds it dry close-mic speech over a synthesised chord, out of distribution on both sides.

Note what the do-nothing baseline has to be. Correlating an output against its **input** is right for a round trip and wrong here, because a network that returned its input unchanged would score beautifully. For separation the meaningful baseline is the unprocessed mixture, measured the same way.

11.2 Separation, really: the check that measures usefulness

On a real recording no stems exist, so no SI-SDR against a source is available and every reading is a **contrast** the mixture is measured under identically. One 19.8-minute stereo stream at 44.1 kHz — a streamer talking over somebody else's music — at 50% overlap-add. `contrast` is the loudest tenth of 250 ms frames against the quietest tenth, with the frames chosen by the **mixture** so all three columns read the same instants; `bed out` is how far the vocals stem sits under the mixture in those quiet frames, which is the music that came out of it.

excerpt	side below mid	partition	vocals rms	instr rms	contrast mix / voc / instr	bed out
900–960 s	18.6 dB	34.9 dB	−2.5 dB	−5.1 dB	19.9 / **24.4** / 10.5 dB	**−6.5 dB**
900–930 s	17.1 dB	36.3 dB	−3.0 dB	−5.2 dB	22.1 / **25.7** / 11.6 dB	−6.0 dB
180–210 s	14.3 dB	34.2 dB	−2.6 dB	−3.5 dB	12.9 / **16.8** / 7.7 dB	−5.5 dB
480–510 s	16.2 dB	34.6 dB	−0.7 dB	−7.3 dB	22.5 / 23.5 / 16.7 dB	−2.0 dB
900–960 s, mono fold	—	37.5 dB	−2.6 dB	−5.4 dB	19.9 / 22.9 / 10.7 dB	−5.1 dB

Figure 7: The real-recording mode. **These numbers and the synthetic ones answer different questions and must not be merged.**

It separates, and the amount is modest. The three contrast columns move together in the way a working separation has to: the vocals stem's contrast comes out **above** the mixture's and the instrumental stem's **below** it, which is one stem following the intermittent speech and the other the continuous bed. But 5–6.5 dB of bed removal is a long way from the 15–20 dB this model reaches on a song, so the music is attenuated rather than gone.

Four things about that table that are easy to get wrong:

- **The 480 s row is a control, not an outlier.** That region's bed stops between phrases instead of running under them, so there is little bed in the quiet frames to remove — and the instrumental stem's contrast rises to 16.7 dB, tracking a bed that is itself intermittent. Less removal where there is less to remove is the model behaving.
- **Read the gaps at 250 ms, not at one second.** A between-sentence gap is a few hundred milliseconds, so at a one-second window no "quiet" frame is speech-free and the verdict **inverts**: the same stems on the same 60 s give 2.1 dB of removal and a vocals contrast **below** the mixture's, which reads as a model that separated nothing.
- **Near-mono is not the explanation.** The side channel sits 14–19 dB under the mid on every excerpt, which removes most of the spatial cue a stereo-native separator would use. Folding to true mono and re-running takes the removal from 6.5 dB to 5.1 dB — real, and far too small to

be the gap. What is left is the material: a speaking voice is not a sung one. Which also means a **mono corpus loses almost nothing** by going through this stage.

- **The partition is 34–37 dB here against 41.5 dB on one chunk.** Overlap-add seams are part of that — the synthetic mode runs a single chunk with no seam at all — but demonstrably not all of it: the mono fold has the same file, the same hop and therefore the same seams, and reads 37.5 dB against the stereo run's 34.9. The partition reading is content-sensitive, which is worth knowing before treating a change in it as a regression.

Every figure above reads the mono downmix, and the stems are stereo. Folding both mixture and stem to mid puts the bed 4.1 dB down in that excerpt's speech gaps; reading the same two files as written gives 1.0 dB, because what the stem keeps of the bed is largely out of phase between the channels and cancels in the fold, where the mixture's own level barely moves. Neither reading is wrong and they are not interchangeable — the mono one is what everything downstream will decode.

11.3 The finding that is not in decibels

Transcribe the same 60 s of mixture and of vocals stem with the same settings, and the decibels turn out to undersell the stage badly:

- **The words are the same** — same content, same order, differing in one character across the minute.
- **The segmentation is not, and that is the finding.** The mixture yields **5** segments, one of them a single merged run of most of a phrase[1] the stem yields **14**, one per utterance. Segmentation cuts on silence (Section 3), and a continuous bed means the recording **has** no silence — so a corpus behind music cannot be cut into sentences at all until the bed comes off.
- Sliced at the default parameters, the same 60 s gives **5** clips holding 58 of its 60 seconds before separation, and **12** holding 39 after. The mixture has no sentence boundaries to find, so it "keeps" almost everything as one lump.
- **Emptying the gaps has a cost.** One of the 14 was a clear recogniser hallucination — English in a Chinese-pinned run — and one was a 0.37 s fragment, both in near-silent frames the mixture's longer segments had swallowed. Anything consuming these stems wants a duration floor. And pin the language: left to detect, the two files disagree about which language they are and the comparison stops meaning anything.

12 The order the decisions come in

Nothing in the phase streams between stages: each reads audio files and writes audio files. That costs a decode and an encode per stage and buys the property the whole phase rests on — any stage's output is a legitimate input to any other, in any order, and a user can look at what came out before spending an hour on the next one. Several of these parameters are ones a user is expected to get wrong on the first attempt, which is not a pipeline to hide inside a single process.

Given that, only two orderings are forced, and both follow from the algorithms rather than from convenience:

- **Analysis first, and again at the end.** Once before anything, because the floor is the parameter everything downstream is a knob on, and because it is the only way to find the recordings that cannot be cut at all. Once at the end, at the very parameters segmentation was run with, to see the clip-length histogram and the silence ratio the trainer will actually meet.

---

[1] Recorded as 18.8 s by one harness and 16.9 s by another; the disagreement is unresolved and is left stated rather than averaged. The segment counts and the words agree everywhere.

- **Separation before everything.** It is the only stage whose absence makes a later one **impossible** rather than merely worse: no silence floor recovers sentence boundaries from a recording that has none. It is also the only stage whose output rate and channel count are not a choice, so a resample belongs after it rather than before.

Everything between those is material-dependent. Speaker filtering only when a second voice survives separation; de-hissing before level matching if the hiss would otherwise set the loudness reading; trimming instead of cutting when the recordings are already one utterance each and only have dead ends.

Sources. The algorithms and every constant are from `audio-kit`'s `slice` (frame grid, hysteresis, the online form, the measured floor), `audio-kit`'s `denoise` and `filter`, `preprocess-core` (`analyze`, `clip`, `trim`, `denoise`, `normalize`, `resample`, `separate`, `diarize`, `embed`), `burn-mdx` (`lib`, `net`, `stft`) and `burn-campplus` (`lib`, `fbank`). Measurements are from `burn-mdx`'s `examples/separate` in both its modes and `preprocess-core`'s `examples/timbre`, which are where those numbers stay current; where this page and a module doc disagree, the module doc is right, because it sits beside the code that would change. Nothing here is estimated.

Three algorithms are described where the code delegates to somebody else's implementation of them, and their sources are the standards rather than this workspace: gated loudness is ITU-R BS.1770 / EBU R128, and the de-hiss and resampling stages are ffmpeg's `anlmdn` and `swresample`. What is measured here is what they do to this material.